

VAJA 3 – Nejc Dolinar

1. AWS CLI

Kaj je AWS CLI

AWS CLI (Command Line Interface) je orodje ukazne vrstice, ki omogoča upravljanje AWS storitev preko ukazov v terminalu. Pred uporabo se konfigurira z access key in secret access key (oboje dobimo v IAM-u). Po konfiguraciji izvajamo ukaze tipa `aws <storitev> <akcija> --parametri`.

Ustvarjanje storitev z AWS CLI

Primer – ustvarjanje S3 bucketa, nalaganje datoteke vanj in seznam bucketov:

```
aws configure                # vpiše access key, secret, regijo
aws s3 mb s3://2026nejcdolinar --region eu-central-1  # naredi bucket
aws s3 cp index.html s3://2026nejcdolinar/             # naloži datoteko
aws s3 ls                                           # seznam bucketov
aws s3 ls s3://2026nejcdolinar/                   # vsebina bucketa
```

Brisanje storitev z AWS CLI

```
aws s3 rm s3://2026nejcdolinar/index.html           # zbrise datoteko v bucketu
aws s3 rb s3://2026nejcdolinar                      # zbrise prazen bucket
aws s3 rb s3://2026nejcdolinar --force               # zbrise bucket z vsebino
```

Prednosti AWS CLI

- Hitrost in učinkovitost – en ukaz nadomesti več klikov v konzoli
- Avtomatizacija – ukaze lahko združimo v skripte za ponavljajoča opravila
- Ponovljivost – isti ukaz daje vedno enak rezultat (idealno za infrastrukturo kot kodo)
- Možnost integracije v CI/CD cevovode
- Bolj natančen nadzor in več parametrov kot v grafični konzoli
- Brez potrebe po brskalniku in prijavi v konzolo

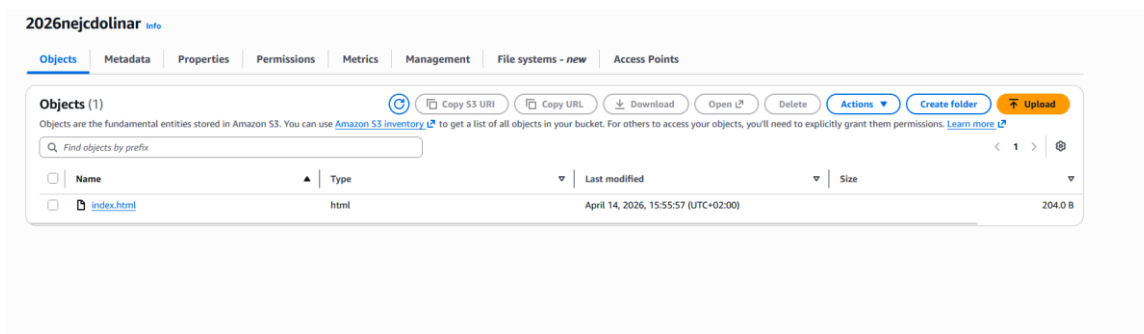
Slabosti AWS CLI

- Strma učna krivulja – treba je poznati strukturo ukazov in parametrov
- Manjši grafični pregled – ni vizualizacije, kot jo nudi konzola
- Tipkarske napake imajo lahko hude posledice (npr. nepričakovan terminate)
- Občutljivi podatki (access keyji) so shranjeni v lokalni konfiguracijski datoteki `~/.aws/credentials` – tveganje, če računalnik ni varovan
- Pri kompleksnih opravilih (npr. CloudFormation, infrastrukturni načrti) so primernejša druga orodja

Dokaz – S3 bucket ustvarjen z AWS CLI

Name	AWS Region	Creation date
 2026nejcdolinar	Europe (Frankfurt) eu-central-1	April 14, 2026, 15:51:21 (UTC+02:00)

S3 bucket 2026nejcdolinar (eu-central-1), ustvarjen 14. 4. 2026 z AWS CLI



Vsebinska bucketa: naložen index.html (204 B) preko AWS CLI

2. IAM (Identity and Access Management)

Ustvarjanje uporabnikov – namen

IAM je AWS storitev za upravljanje identitet in pravic dostopa. V njej ustvarjamo uporabnike, skupine, vloge in pravila (policies). Glavni razlogi za ustvarjanje IAM uporabnikov:

- Vsaka oseba ali aplikacija ima svoje poverilnice (ne deli računa)
- Sledljivost – v dnevnikih (CloudTrail) vidimo, kdo je kaj naredil
- Princip najmanjših privilegijev – vsak dobi samo to, kar res potrebuje
- Hitro odvzemanje pravic (npr. ko zaposleni odide)
- Programatični dostop ločen od konzolnega

Primeri uporabe

- Razvijalec, ki potrebuje dostop do S3 in EC2, dobi uporabnika z ustreznimi pravicami
- Aplikacija, ki nalaga datoteke v S3 bucket, dobi programatični uporabnik z access keyji
- CI/CD strežnik dobi IAM vlogo z dovoljenji za deploy
- Administrator dobi uporabnika z MFA in širšimi pravicami

Pravilo za nastavljanje pravic

Glavno pravilo je princip najmanjših privilegijev (least privilege): uporabnik dobi samo tiste pravice, ki jih nujno potrebuje za svoje delo, in nič več. Tako se v primeru kompromitacije zmanjša škoda. Pravice damo preko policies, ki jih priložimo neposredno uporabniku ali skupini, v kateri je.

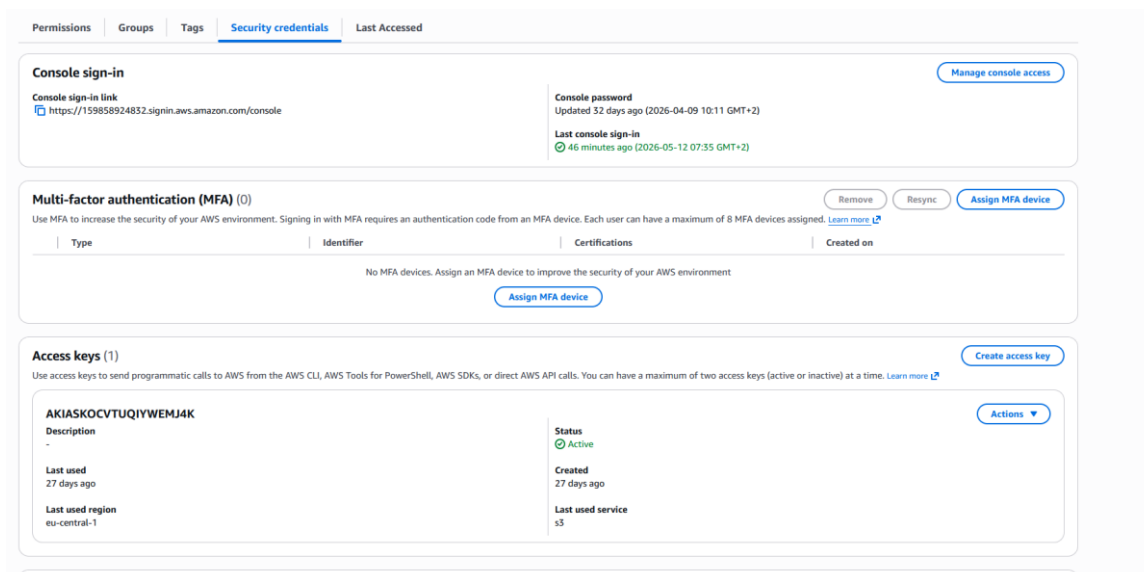
Koliko in katere pravice (splošno)

- Za začetnike in testiranje pogosto uporabljamo AWS managed policies (npr. AmazonS3FullAccess, AmazonEC2ReadOnlyAccess)
- Za produkcijske scenarije pišemo lastne (customer managed) policies, ki natančno določajo dovoljene akcije in vire
- Ne uporabljamo root računa za vsakodnevno delo – samo za začetno nastavitvev
- Vsako pravico utemeljimo – zakaj jo ta uporabnik potrebuje

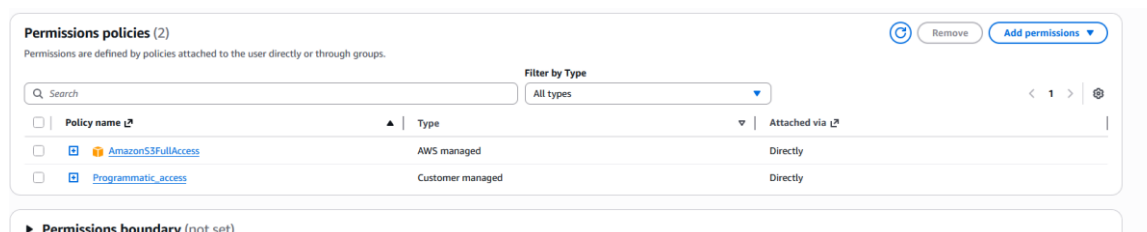
Dokazi – IAM uporabnik programmatic

☐	programmatic	/	0	46 minutes ago	32 days	46 minutes ago	Active - AKIASMOCVTU...	27 days	27 days ago	arn:aws:iam::159858924832:user/prog...
--------------------------	--------------	---	---	----------------	---------	----------------	-------------------------	---------	-------------	--

IAM uporabnik »programmatic« – aktiven, z Access key ARN arn:aws:iam::159858924832:user/prog...



Security credentials uporabnika: konzolni dostop, Access keys (programmatic), zadnja uporaba pred 27 dnevi v eu-central-1 za S3



Permissions policies uporabnika: AmazonS3FullAccess (AWS managed) in Programmatic_access (Customer managed)

3. VPC (Virtual Private Cloud)

Kaj je VPC

VPC je navidezno privatno omrežje v AWS oblaku, ki ga izoliramo od drugih strank. Znotraj VPC-ja definiramo lasten naslovni prostor, podomrežja (subnete), usmerjevalne tabele, internetne prehode in firewallle. Vsak AWS račun ima privzeti VPC v vsaki regiji (npr. 172.31.0.0/16), lahko pa ustvarimo poljubno število svojih.

Kako naredimo VPC

- V AWS konzoli: VPC → Your VPCs → Create VPC
- Določimo ime in IPv4 CIDR blok (npr. 10.0.0.0/16)
- Izberemo tenancy (Default ali Dedicated)
- Po želji dodamo IPv6 in tagje
- Po izdelavi VPC ustvarimo še subnete, route table in po potrebi Internet Gateway

CIDR – kaj določamo s CIDR zapisom

CIDR (Classless Inter-Domain Routing) zapis določa obseg IP naslovov, ki pripadajo omrežju. Zapis je v obliki IP/prefix, kjer prefix pove, koliko bitov je omrežnih (fiksni), preostali biti pa določajo posamezne naslove znotraj omrežja.

- Več kot je bitov v prefiksu, manjše je omrežje (manj naslovov)
- /16 → 65.536 naslovov, /24 → 256 naslovov, /27 → 32 naslovov
- Formula: število naslovov = $2^{(32 - \text{prefix})}$

- AWS dovoljuje CIDR prefikse od /16 (največje) do /28 (najmanjše)

Razlaga primera 10.0.2.16/27

Prefix je /27, kar pomeni 27 omrežnih bitov in 5 bitov za naslove znotraj omrežja:

- Skupno število naslovov: $2^5 = 32$
- Začetek omrežja (network address): 10.0.2.16
- Konec omrežja (broadcast): 10.0.2.47
- Razpon: 10.0.2.16 – 10.0.2.47

V AWS subnetu je 5 naslovov rezerviranih (prvi 4 + zadnji), zato je za naše instance uporabnih 27 naslovov:

- 10.0.2.16 – network (rezerviran)
- 10.0.2.17 – VPC usmerjevalnik (rezerviran)
- 10.0.2.18 – DNS (rezerviran)
- 10.0.2.19 – rezervirano za prihodnjo uporabo
- 10.0.2.20 – 10.0.2.46 – uporabni naslovi (27)
- 10.0.2.47 – broadcast (rezerviran)

Dokazi – VPC v AWS

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR	DHCP option set	Main route table
backup-system-vpc	vpc-0b9b456048a8b6dc	Available	-	-	Off	10.0.0.0/16	-	dhcp-0f14cf5632ad7367e	rtb-0a3689b6314a7737
-	vpc-055845429c865406c	Available	-	-	Off	172.31.0.0/16	-	dhcp-0f14cf5632ad7367e	rtb-02a3689b6314a7737

Seznam VPC-jev: backup-system-vpc (10.0.0.0/16) in privzeti VPC (172.31.0.0/16)

Detalji VPC-ja vpc-055845429c865406c: CIDR 172.31.0.0/16, Resource map prikazuje VPC, subnet vaja 2, route table in Internet Gateway

4. Subnet (podomrežje)

Namen podomrežij

Subnet je razdelek znotraj VPC-ja z lastnim CIDR blokom, ki je podmnožica VPC CIDR-ja. Subneti služijo za:

- Logično ločevanje različnih delov omrežja (npr. web sloj, baza, management)
- Razdelitev po Availability Zone (vsak subnet je v eni AZ – za visoko razpoložljivost)

- Določanje, kateri viri so dostopni iz interneta in kateri ne (javni vs. privatni)
- Lažje upravljanje varnostnih pravil in usmerjanja prometa

Kako naredimo subnet

- V AWS konzoli: VPC → Subnets → Create subnet
- Izberemo VPC, v katerem bo subnet
- Določimo ime, Availability Zone in IPv4 CIDR (npr. 10.0.1.0/24)
- Po želji prilagodimo še druge nastavitve in tagje

Kaj moramo nastaviti

- CIDR blok subneta – mora biti podmnožica VPC CIDR-ja in se ne sme prekrivati z drugimi subneta
- Availability Zone – fizična lokacija znotraj regije
- Route Table – usmerjevalna tabela, ki določa, kam gre promet iz subneta
- Auto-assign public IPv4 – ali instance v subnetu samodejno dobijo javni IP

Kako naredimo subnet javen (public)

Javni subnet je tisti, ki ima neposredno povezavo z internetom:

1. V VPC-ju ustvarimo Internet Gateway (IGW) in ga priključimo VPC-ju
2. Ustvarimo Route Table za javni subnet
3. V Route Table dodamo ruto: 0.0.0.0/0 → naš IGW
4. Route Table povežemo s subnetom (Subnet associations)
5. V subnet nastavitvah omogočimo Auto-assign public IPv4

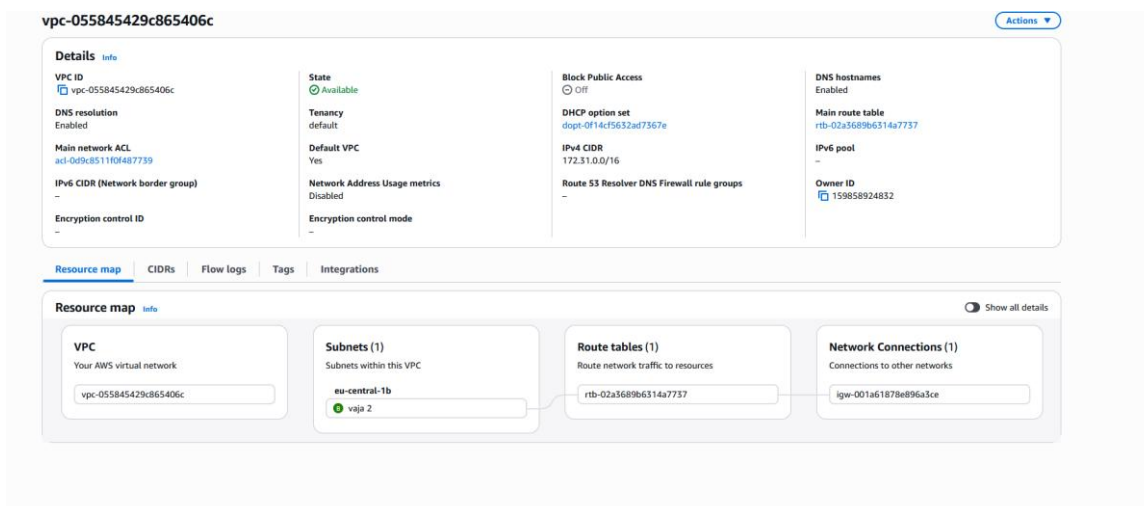
Kako naredimo subnet privaten (private)

Privatni subnet nima neposredne povezave z internetom – instance v njem imajo samo privatne IP-je:

1. Subnetu pripišemo Route Table brez 0.0.0.0/0 → IGW pravila
2. Promet znotraj VPC (lokalna ruta) deluje normalno
3. Za izhodni dostop do interneta (npr. posodobitve) lahko uporabimo NAT Gateway v javnem subnetu in dodamo ruto 0.0.0.0/0 → NAT GW
4. Auto-assign public IPv4 mora biti izklopljen

Tipična uporaba: spletni strežniki v javnem subnetu, baze podatkov v privatnem.

Dokaz – subnet znotraj VPC-ja



Resource map VPC-ja: viden je subnet »vaja 2« v eu-central-1b, povezan z route table in Internet Gateway-jem (javni subnet)

5. Kriptografski ključ – .pem vs .ppk

Razlika med .pem in .ppk

.pem (Privacy Enhanced Mail) je standardni format za OpenSSH ključke, ki ga uporabljajo Linux, macOS in tudi Windows OpenSSH (vključno s scp.exe). Datoteka je v Base64 kodiranju z glavo »-----BEGIN RSA PRIVATE KEY-----«. AWS ga ponudi privzeto pri ustvarjanju key paira.

.ppk (PuTTY Private Key) je lasten format programa PuTTY za Windows. Uporabljajo ga PuTTY, PuTTYgen, WinSCP in podobni Windows klienti. Iz .pem v .ppk pretvorimo s PuTTYgen-om (Load → Save private key), iz .ppk v .pem pa s PuTTYgen → Conversions → Export OpenSSH key.

Vsebina obeh datotek je isti privatni ključ, le format zapisa je drugačen. Javni del je pri AWS-u shranjen ločeno na strežniku.

Kam shranimo javni ključ

Javni ključ se shrani na strežnik, do katerega se želimo prijavljati. Na Linuxu (npr. EC2 z Ubuntu) je shranjen v datoteki:

```
~/.ssh/authorized_keys
```

Za uporabnika ubuntu na EC2 instanci je to /home/ubuntu/.ssh/authorized_keys. AWS pri zagonu instance javni ključ iz izbranega key paira samodejno doda v to datoteko. Sicer ga lahko dodamo ročno z ssh-copy-id ali z direktnim urejanjem datoteke.

Kam shranimo privatni ključ

Privatni ključ ostane samo pri odjemalcu (na našem računalniku). Nikoli ga ne nalagamo na strežnik in nikoli ga ne delimo z nikomer. Tipične lokacije:

- Linux/macOS: ~/.ssh/ime_kljuca.pem z dovoljenji 600 (chmod 600)
- Windows: poljubna varna mapa (npr. C:\Users\Nejc\.ssh\ali E:\Downloads\Web Downloads\)

Datoteka mora biti dostopna samo lastniku. Na Linuxu lahko brez dovoljenj 600 SSH klient zavrne uporabo ključa.

Dokaz – oba formata ključa na lokalnem računalniku

Vaja3.pem	12. 05. 2026 08:24	PEM File	2 KB
Cloud-vaja2	12. 05. 2026 08:18	Microsoft Word D...	409 KB
vaja2	12. 05. 2026 07:37	PuTTY Private Key...	2 KB

Mapa z obema formatoma istega ključa: Vaja3.pem (OpenSSH) in vaja2.ppk (PuTTY)

6. Policy (pravila dostopa)

Kaj je policy

Policy je dokument v JSON formatu, ki natančno opisuje, katere akcije so dovoljene ali zavrnjene na katerih AWS virih. Ko policy priložimo IAM uporabniku, skupini ali vlogi, ta dobi vse v njej opisane pravice.

Zakaj smo pri dostopu do AWS CLI uporabili policy

Ko se preko AWS CLI prijavimo z access key in secret access key, AWS na podlagi teh poverilnic identificira IAM uporabnika. Ali sme uporabnik izvesti določen ukaz (npr. aws s3 mb), odloča policy, priložen temu uporabniku. Brez ustreznega policyja vsak ukaz CLI vrne napako AccessDenied, čeprav je avtentikacija uspela.

V vaji smo programatskemu uporabniku priložili policy, ki mu je dovolil delati s S3 in drugimi storitvami, ki smo jih potrebovali za vajo. Brez policyja bi CLI ukazi spodleteli.

Kaj vse nastavimo v policy

Policy dokument vsebuje naslednja polja:

- **Version:** različica jezika policy (vedno »2012-10-17«)
- **Statement:** seznam pravil – vsako pravilo lahko vsebuje:
 - Sid – opisno ime pravila (opcijsko)
 - Effect – Allow ali Deny
 - Action – katere akcije (npr. s3:GetObject, ec2:*)
 - Resource – na katerih virih (npr. ARN bucketa ali *)
 - Condition – dodatni pogoji (npr. samo z določenega IP-ja, ob določeni uri)
 - Principal – kdo (uporablja se predvsem v resource-based policies)

Pravila se preverjajo kumulativno – eksplicitni Deny vedno preglasi Allow.

Dokazi – ustvarjen policy in njegova vsebina

Policy name	Type	Used as	Description
Programmatic_access	Customer managed	Permissions policy (2)	-

Seznam IAM policies: Programmatic_access (Customer managed), uporabljen kot Permissions policy

Programmatic_access info

Policy details

Type Customer managed	Creation time March 26, 2026, 16:51 (UTC+01:00)	Edited time March 26, 2026, 16:51 (UTC+01:00)	ARN arn:aws:iam::159858924832:policy/Programmatic_access
--------------------------	--	--	---

Permissions | Entities attached | Tags | Policy versions (1) | Last Accessed

Permissions defined in this policy info

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

```

1- [
2-   {
3-     "Version": "2012-10-17",
4-     "Statement": [
5-       {
6-         "Sid": "TemporaryAccess",
7-         "Effect": "Allow",
8-         "Action": "*",
9-         "Resource": "*"
10-      }
11-    ]
12-  }

```

Copy Edit Summary JSON

JSON vsebina policyja *Programmatic_access*: *Effect Allow*, *Action **, *Resource ** – polni dostop (»*TemporaryAccess*«)

7. Git in GitHub

Kaj je Git

Git je porazdeljen sistem za upravljanje različic (version control system) izvorne kode. Razvil ga je Linus Torvalds leta 2005 za potrebe razvoja Linux jedra. Git beleži zgodovino sprememb v projektu (commits), omogoča delo več razvijalcev hkrati (branches, merge) in vrnitev v starejše različice.

Primeri uporabe

- Spremljanje sprememb v izvorni kodi skozi čas
- Sodelovanje več razvijalcev na istem projektu
- Vejanje (branching) za razvoj funkcionalnosti brez vpliva na glavno vejo
- Vrnitev k starejši različici, če nekaj pokvarimo
- Postavljanje izdaj (releases) in označevanje različic z oznakami (tags)
- CI/CD – ob vsakem push-u se sproži samodejno gradnja in objava

Osnovni Git ukazi

Ustvarjanje novega repozitorija:

```

git init                # inicializira repozitorij v trenutni mapi
git add .               # doda vse spremembe v staging
git commit -m "prvi commit" # potrdi spremembe v lokalno zgodovino

```

Nalaganje na GitHub:

```

git remote add origin https://github.com/nejc/repo.git
git branch -M main      # preimenuje vejo v main
git push -u origin main  # pošlje na GitHub in nastavi sledenje

```

Kloniranje obstoječega repozitorija z GitHub:

```
git clone https://github.com/nejc/repo.git
```

Prenos sprememb iz GitHub:

```

git pull                # potegne najnovejše spremembe iz remote
git fetch               # le preveri spremembe brez zlivanja

```

Hranjenje nove verzije (po spremembah):

```

git status              # vidi, kaj se je spremenilo
git add .               # doda spremembe v staging
git commit -m "opis spremembe" # ustvari novo verzijo

```


git push

pošlje spremembo na GitHub

Kako kopiramo repozitorij na strežnik (GitHub) – postopek

- 1. Na GitHub.com kliknemo »New repository« in določimo ime ter vidnost (public/private)
- 2. Na lokalnem računalniku se postavimo v mapo s projektom
- 3. Inicializiramo Git: git init
- 4. Dodamo datoteke v staging: git add .
- 5. Ustvarimo prvi commit: git commit -m "prvi commit"
- 6. Povežemo lokalni repozitorij z GitHub remote: git remote add origin <URL>
- 7. Pošljemo na GitHub: git push -u origin main
- Pri novjših GitHubih je za push potrebna avtentikacija s personal access tokenom (PAT) ali SSH ključem, ne več z geslom.

V mojem primeru sem uporabil GitHub Desktop, kar grafično vodi skozi enake korake (Add → Publish repository → push).

Povzetek

V Vaji 3 sem se naučil uporabljati AWS CLI za upravljanje storitev iz ukazne vrstice (ustvarjanje in brisanje S3 bucketov), ustvariti IAM uporabnika s programatičnim dostopom in mu prirediti policies za nadzor pravic, ustvariti VPC z lastnim CIDR blokom in razumeti delitev na javne in privatne subnete, razumeti razliko med .pem in .ppk formatom ključa ter njuno pravilno shranjevanje, ter osnove Git in GitHub za upravljanje različic kode.